

**FACULTY
OF MATHEMATICS
AND PHYSICS**
Charles University

MASTER THESIS

František Dostál

**Fitness and novelty in evolutionary
reinforcement learning**

Name of the department

Supervisor of the master thesis: Mgr. Roman Neruda, CSc.

Study programme: Mgr.

Study branch: Artificial Intelligence

Prague 2023

I declare that I carried out this master thesis independently, and only with the cited sources, literature and other professional sources. It has not been used to obtain another or the same degree.

I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In date
Author's signature

Dedication.

Title: Fitness and novelty in evolutionary reinforcement learning

Author: František Dostál

Katedra teoretické informatiky a matematické logiky: Name of the department

Supervisor: Mgr. Roman Neruda, CSc., department

Abstract: Novelty is a novel approach to modeling selection criteria in evolutionary algorithms and has been proven as viable technique of avoiding pitfalls of false optima in tasks abundant with them, such as solving mazes. Rather than closing the topic however, this finding opened other problems to explore: How to properly apply novelty in tasks that yield slightly better to conventional approaches? How to properly model behavioral space necessary for novelty computation? In this thesis we investigate use of novelty in selected reinforcement learning tasks, its combinations with classical fitness and propose behavior space models for the respective RL tasks.

Keywords: evolution novelty fitness behavioral space

Contents

Introduction	2
1 Introduction to Artificial Agents	3
1.1 Agents and enviroment	3
1.2 Neural Network Agent	4
2 Enviroments	5
2.1 Reinforcement learning	5
2.2 Gymnasium	5
2.2.1 Cartpole	6
2.2.2 Lunar Lander	6
3 Evolutionary algorithms	7
3.1 Continuous Optimalisation	8
3.1.1 Evolutionary Strategies [Beyer and Schwefel, 2002]	8
3.1.2 Differential Evolution	9
3.2 Utility functions	9
3.2.1 Fitness	9
3.2.2 Pure Novelty	9
3.2.3 Combinations of Novelty	10
4 Comparison criteria and conditions	11
4.1 General criteria	11
4.2 Conditions	11
4.3 Individual hyperparameter selection	11
4.4 $\mu + \lambda$ hyperparameter selection	11
4.4.1 mutation sigma vs. generations	12
4.5 differential hyperparameter selection	12
5 Experiments	14
5.1 Hyperparameter selection	14
5.1.1 Cartpole	14
5.1.2 Lunar Lander	14
5.2 Final comparison	14
Conclusion	15
Bibliography	16
List of Figures	17
List of Tables	18
List of Abbreviations	19
A Attachments	20
A.1 First Attachment	20

Introduction

1. Introduction to Artificial Agents

To understand the reasoning guiding this work we have to first look at the fundamental subject of our efforts - the artificial agents.

We will look at their structure, guiding principles, practical implementation and nor last nor least, their use.

1.1 Agents and enviroment

All agents exists in enviroments that provide context to their agency. An agent gets information about the **enviroment** it find itself in (e.i. **percepts**) through **sensors** and acts on the enviroment through **actuators**. Both actuators and sensors are usually characteristics of the agent. Another characteristic of an agent would be setup of its internal machinations.

Definition 1. [Russell and Norvig, 2021] Let E be enviroment where agent G is located. G is equiped with actuators A and sensors P and so we can identify $obss_P(E)$, a space of all possible sequences of percepts available to G within E and $A(E)$ space of available actions in E . Then function $G : obss_P(E) \rightarrow A(E)$ is called an agent function, defining behavior of the agent G in enviroment E .

We ilustrate relationship of these concepts in the diagram 1.1. Because we want our agents to solve some tasks in their respective enviroments with certain efficiency i.e. we want them to be intelligent, we need to define a notion of rationality that can help us better capture what we mean by intelligence. First however we need to measure how usefull or detrimental a situation can be for our agent.

Definition 2. [Russell and Norvig, 2021] Be that E is an enviroment and $seq(E)$ is a space of all possible sequences of states within E . Then function $f : seq(E) \rightarrow \mathbf{R}$ is a performance measure of some agent operating in E .

Definition 3. [Russell and Norvig, 2021] For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

We should note that in the real-world applications the rationality of an agents can often be limited by their capacity to store the evidence gathered from percepts, by the lack of their built-in knowledge or the computational capacity to

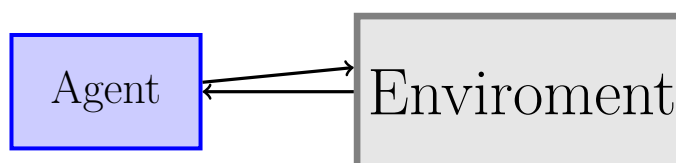


Figure 1.1: Agent and enviroment realtionship

estimate action sequence that can be expected to achieve global maximum of the agent's performance measure.

Since our focus is mainly on the evolutionary algorithms in the scope of this thesis we will only encounter the most simple of agents subject to the most severe of such limitations.

Definition 4. [Russell and Norvig, 2021] *An agent is called simple reflex agent if it takes into account only the current percepts, never keeping any history or state information during it's run.*

1.2 Neural Network Agent

The concept of neural networks (NN) comes from an area of machine learning called deep learning that was conceived to attempt to model biological neuron activity. Since it has become one of the most successful branches of machine learning. [Russell and Norvig, 2021] The neural networks consist of neurons organised in layers.

Definition 5. *Let $W \in R^{m \times n}$ and $b \in R^n$ and let $f : R^n \rightarrow R$. Then we call W the weights of a neuron layer with n neurons, b the bias of the layer and f the activation function of the layer when given an input vector $x \in R^m$ we calculate the output of the layer as $f(W^T x + b)$. Two layers have directional connection when the output of first layer is used as input of the subsequent layer.*

We recognise several typical activation functions:

- sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- tanh

$$\tanh(x) = \frac{e^{2x} - 1}{e^{2x} + 1}$$

- ReLU

$$ReLU(x) = \max(0, x)$$

Definition 6. [Russell and Norvig, 2021] *A neural network where the connections between layers of neural network form an acyclic graph is called a feed-forward neural network.*

Because the feed-forward NN doesn't hold any state or holds onto any previously acquired results, only produces output for a given input, they are ideal for implementation of simple reflex agents.

2. Enviroments

2.1 Reinforcement learning

Reinforcement learning might be best introduced by description of the problem it aims to solve.

Definition 7 (Markovov's decision problem). *We define Markovov's decision problem (MDP) by a fourtuple (S, A, P, R) where:*

- *S is a set of all possible states of the system*
- *A is a set of all possible actions available in the system*
- *P is a transition function, $P(s, a, s')$ being probability of transition from s to s' give action a is exectuted.*
- *R is reward function, $R(s, a, s')$ being the reward recieved when the execution of action a in state s leads to state s' .*

To tie this into the information from chapter 1 $A = A(E)$, R is performance measure of an agent solving the MDP and the percepts correspond to real states of the enviroment. This would not be the case for partially observable MDP.

Definition 8. *POMDP*

Definition 9. *Best policy*

The goal of reinforcement learning is then to aproximate best policy by trial and error, based on the recieved reward, as accuratelly as possible. This can be done utilising various methods such as Q-learning, but imporatantly for this thesis it could be

2.2 Gymnasium

In this thesis we will set our agents into enviroments provided by Gymnasium project which emerged from it's predecessor OpenAI Gym project. Both of them Python libraries, trying to provide researches with benchmark reinfroment learning problems in form of simple games, sometimes even recreating old console games such as Air Raid or Backgammon.

While Gymnasium provides wide range of enviroments, quite a number of them if not most do not provide their outputs in structured form. Instead the resulting screen has to be processed by user defined visual processor to extract infomation crucial for completing the task.

This introduces a layer of complexity that is outside of the scope of this thesis. Therefore we have decided to narrow our selection only to enviroments which do provide structured output.

2.2.1 Cartpole

Cartpole is one of the most known benchmark environments in the Gymnasium library. It's a simple environment where the agent controls a cart rolling on flat surface with a pole on top. The goal is to keep the pole balanced on top of the cart. The reward is calculated every step depending on the angle.

2.2.2 Lunar Lander

Lunar Lander is somewhat more complex domain to optimise for than the Cartpole. Every step the reward is calculated based on several criteria, as opposed to only an angle of a stick, controlling the landers descend and additional points are awarded for completing certain tasks. Additionally there are several variables determining the environments behaviour during evaluations such as the strength of wind. Unlike in cartpole where

3. Evolutionary algorithms

The evolutionary algorithms (EA) are a class of optimisation methods modeled after our ideas of biological process of evolution.

Definition 10. *General optimisation problem takes the form:*

$$\begin{aligned} \text{minimise:} \quad & f(x) \\ \text{subject to:} \quad & g_i(x) \leq 0, \quad i = 1, \dots, m, \\ & h_j(x) = 0, \quad j = 1, \dots, p, \end{aligned}$$

where x is the solution vector, f is objective function, and g_i, h_j are constraints imposed on x

Often in literature on EA we might find confluence of the term objective function and another term borrowed from biology - fitness. For reasons that will become apparent in the course of this chapter we will make sharp distinction between them for the purposes of this thesis. By objective function we will understand function for which the algorithm directly optimises while fitness will be used to describe semantic quality of an individual in the context of its use. Because of this biological connection solution vectors are in the literature often referred to as individuals and we will refer to them this way henceforth as well. Alternatively when emphasising either the numerical form of the vector or the semantical interpretation of this form beyond the optimisation algorithm the individual may be referred to as either a genotype or phenotype respectively. Group of individuals representing candidate solutions at a given time is referred to as a population. Evolutionary algorithms then evolve the population by applying three operations to it in a loop until either the population converges to optimum or the limit on number of steps is reached. Crossover usually involves recombina-

Algorithm 1: Generalised Evolutionary Algorithm

```
Data: Initial population = pop
Data: N = Number of generations
Result: Optimised population = pop
for  $g \in 1..N$  do
     $co \leftarrow \text{Crossover}(pop);$ 
     $m \leftarrow \text{Mutate}(co);$ 
     $pop \leftarrow \text{Select}(m);$ 
end
return  $pop$ 
```

tion of features from two or more individuals from the population. Its purpose is to provide means of exploitation of features beneficial to the population already developed by some individual.

On the other hand mutation provides means of exploration by introducing randomness into the individuals.

EA without crossover would be more akin to a random walk algorithm. Selection

then is about filtering the resulting individuals based on the objective function of the algorithm. It might be performed deterministically or with varying degree of stochasticity. One of the semi-stochastic methods is tournament selection

Algorithm 2: Torunament selection

Data: Population = pop
Data: N = Number of tournament participants
Result: Selected victor = v
 $co \leftarrow \text{CrossOver}(pop);$
 $m \leftarrow \text{Mutate}(co);$
 $pop \leftarrow \text{Select}(m);$
return pop

3.1 Continuous Optimalisation

Historically the individuals optimised by EA used be primarily vectors over binary domain - $\{0,1\}$. However The form then also depends on the type of algorithm chosen. While some CO algorithms may have their analog in other field of evolutionary algorithms, others might not be transferable outside of the domain. The literature shows many aproaches without any apriori limit to their variety. In this thesis we have will be refering to two most typical aproaches

3.1.1 Evolutionary Strategies [Beyer and Schwefel, 2002]

Evolutionary strategies (ES) are very direct incarnation of algorithm 1 into the realm of real numbers. Distinct feature of evolutionary staregies is the mutation operator that takes as a parameter standard devaition σ and which mutates the value stochastically by a number $y \sim N(0, \sigma)$ We recognise two types of evolutionary strategies: $\lambda + \mu$ and λ, μ .

Algorithm 3: Evolutionary Strategy $\lambda + \mu$

Data: population count, offspring count, number of generations
Result: optimised population
 $\lambda \leftarrow \text{population count};$
 $\mu \leftarrow \text{offspring count};$
 $pop \leftarrow \text{InitPop}(\lambda);$
for $g \in 1..N$ **do**
 $off \leftarrow \text{CrossOver}(pop, \mu);$
 $off \leftarrow \text{Mutate}(off);$
 $pop \leftarrow \text{Select}(pop + off, \lambda);$
end
return pop

Algorithm 4: Evolutionary Strategy λ, μ

Data: population count, offspring count, number of generations

Result: optimised population

$\lambda \leftarrow$ population count;

$\mu \leftarrow$ offspring count ;

$pop \leftarrow InitPop(\lambda)$;

for $g \in 1..N$ **do**

$off \leftarrow CrossOver(pop, \mu)$;

$off \leftarrow Mutate(off)$;

$pop \leftarrow Select(off, \lambda)$;

end

return pop

3.1.2 Differential Evolution

3.2 Utility functions

3.2.1 Fitness

Using objective function as the utility function of the algorithm is the most direct way of establishing relationship between the solution we want and the individuals in the evolving population.

Though this approach ensures that the most optimal individuals from given population will generally be selected into the next generation during the evolution it does not guarantee that their potential for evolving is not diminished either. This way the evolution using fitness can easily get stuck in local optimum of the objective function. While in most cases it might be still possible to reach global optimum on the given problem by changing the hyperparameters to favor random exploration, it may come to point where the benefits of evolutionary approach are made void.

3.2.2 Pure Novelty

In the paper ? a new approach to utility functions has been proposed, called novelty, designed to help avoid traps of local optima.

Definition 11. [Doncieux et al., 2019] Let E be enviroment and S be a space of all possible states of the enviroment and let $d(x, y)$ be some metric. Then (B, d) is a behavioral space and a function $o_B : S^T \rightarrow B$ is called observer function.

Definition 12. [?] Let P be a population of individuals within EA and $(Img(B), d)$ be a behavioral space. Then for each individual a we define novelty as such:

$$N(a) = \frac{\sum_{b \in P, b \neq a} d(a, b)}{|P| - 1}$$

With novelty as utility function the selection process incentivises behaviours inversely to their representation in the current population, in other words according to how "novel" they are. This allows the evolutionary search to retain variety

within the population across generations, leveraging the recombination as well as mutations to leave local optima.

Downside of this approach is in the ambiguity in the choice of behavioral space. Incorporating too many environment variables will lead to the curse of dimensionality. Leaving out an important variable means the algorithm will not retain diversity where it matters, making novelty pointless. This means the behavioral function and space have to be selected empirically for given problem.

3.2.3 Combinations of Novelty

The novelty does not take into account the individual's performance with objective function. Therefore when two individuals with similar behavioral characteristics appear but one of them performs significantly better than the other, pure novelty algorithm can easily select the less performant individual, losing the more performant genome to further generations. To prevent this we want to meaningfully combine novelty with fitness.

4. Comparison criteria and conditions

To compare different types of guiding functions well, we have to select criteria by which we compare them and establish conditions under which will this comparison take place so that our conclusions are fair but not misleading.

4.1 General criteria

Since fitness provides direct measure of performance we would like to make our comparison on the fitness of the final population and its distribution in it. To that end we have to perform experiments with variety of initialisations of the respective environments, getting more comprehensive picture then from a single run. This will be mainly reflected in the seed for random numbers generator of the environment. Since the promise of the novelty approach lies mainly in avoiding the trap of local optima we also want to review statistics of each generation produced by the algorithm runs

However we also have to take into account the hyperparameters with which these populations have been achieved. This is especially (but not limited to) population size and number of generations, hyperparameters responsible for most of the computational complexity of a evolutionary algorithm run.

4.2 Conditions

The conditions are defined by the hyperparameters of the algorithms and the hyperparameters of the individuals.

4.3 Individual hyperparameter selection

The hyperparameters characterising individuals are number of layers and the number of neurons in their layers. The choice of these parameters is specific for each task. Generally the more complex is the task the more of either layers or neurons per layer the handling NN requires to be suitable for it. We determine these parameter based on empirical experience with the tasks as optimising this choice is beyond the scope of this thesis.

4.4 $\mu + \lambda$ hyperparameter selection

The algorithm hyperparameters will be selected in a series of grid searches, tailored to each environment and algorithm type. While it is theoretically possible to setup one large comprehensive gridsearch for each algorithm, lacking specialised hardware, the exponential dependence on number of hyperparameters would make it highly impractical in reality.

evaluation episodes vs. generations

Before we start other experiments we have to determine how many episodes will be used to evaluate an individual during the algorithm's run during these experiments. This parameter directly influences how much information the algorithm has extracted about the individual, as more episodes will lead to more indicative behavioural descriptors. However since the episodes take up nontrivial part of the computation we want to limit them as much as we can.

4.4.1 mutation sigma vs. generations

This experiment will help us narrow our choice of sigma used for the continuous mutation distribution. Because the mutations are commutative we can infer that many small mutations will add up to fewer larger mutations and vice versa the larger mutation can be emulated by several smaller mutations. The number of mutations applied on individual is limited by number of generations the algorithm runs. Therefore we need to balance our choice of sigma so that the mutations do not disturb progression of our population and but also achieve beneficial change during as few generations as possible.

offsprings vs. mutation-rate

This experiment should help us find meaningful mapping between values of mutation rate and size of the offspring pool. The mutation rate determines how much random information is being introduced to the population while larger offspring pool allows for more effective harnessing of this random fluctuations. During the experiment the population pool is static and cross-over is eliminated.

population vs. cross-rate

This experiment should help us find meaningful mapping between values of cross-over rate and the size of the population pool

definitive parameter search

In this experiment we use previously acquired mappings to search for the most performative combination of the hyperparameters.

4.5 differential hyperparameter selection

population vs. gamma learning rate

We can interpret the learning rate as probabilistic constant, a measure of belief in the direction calculated from the differential mutation. We should consider this parameter in contrast with the size of the population as the increasing size might provide better opportunities but also drown them out in less optimal alternatives.

population vs. alpha recombination

We can interpret α as a measure of our belief in suitability of the current population even more directly than the learning rate. Therefore we have to also consider it against the size of this population as the increasing population might provide more optimal individuals out of pocket but their influence on the population direction may ultimately dissolve in the amount of combinations.

population vs. generations

Having learned the best parameter for each population we should consider them in contrast to how many generations are required for the evolution to find good enough solutions.

5. Experiments

5.1 Hyperparameter selection

5.1.1 Cartpole

5.1.2 Lunar Lander

5.2 Final comparison

Conclusion

Bibliography

Hans-Georg Beyer and Hans-Paul Schwefel. Evolution strategies - a comprehensive introduction. *Natural Computing*, 1:3–52, 03 2002. doi: 10.1023/A:1015059928466.

Stephane Doncieux, Alban Laflaquière, and Alexandre Coninx. Novelty search: a Theoretical Perspective. In *GECCO '19: Genetic and Evolutionary Computation Conference*, pages 99–106, Prague Czech Republic, France, July 2019. ACM. doi: 10.1145/3321707.3321752. URL <https://hal.science/hal-02561846>.

Stuart Russell and Peter Norvig. *Artificial Intelligence, Global Edition A Modern Approach*. Pearson Deutschland, 2021. ISBN 9781292401133. URL <https://elibrary.pearson.de/book/99.150005/9781292401171>.

List of Figures

1.1 Agent and enviroment realtionship	3
---	---

List of Tables

List of Abbreviations

A. Attachments

A.1 First Attachment